An AI-Based Advisory System for Sustainable Organic Farming: Crop Recommendation, Leaf Disease Detection and Irrigation Support in One Web Platform

Role	Name	Roll No.	Designation / Department
Student	Sama Manmadh Reddy	23EG107E53	B.Tech, Dept. of AI and ML
Student	Ramaswamy Akshitha	23EG107E51	B.Tech, Dept. of AI and ML
Student	Cheekati Ramesh Kumar Reddy	23EG107E12	B.Tech, Dept. of AI and ML
Project Guide	S. Anitha	—	Assistant Professor, Dept. of AI and ML

Anurag University, Hyderabad, India

Abstract. Most farmers do not have an agronomist on call. They still have to decide what to sow, how to feed the soil and when to water, and one wrong guess can cost a whole season. This paper describes a web application for farmers who prefer organic methods. The user enters soil readings (N, P, K and pH) together with local temperature, humidity and rainfall, and the system proposes suitable crops and natural fertilisers. A photo of a diseased leaf is classified by a convolutional neural network and matched to an organic remedy. The weather forecast is used to say whether a field needs water today, and a chatbot handles everyday questions in plain words. We set out the design, the planned models, the knowledge base behind the advice and the way we intend to test it. Our aim is modest: better yields with less damage to soil and water.

Keywords: organic farming, crop recommendation, leaf disease detection, transfer learning, irrigation scheduling, chatbot, decision support.

I. INTRODUCTION

A smallholder farmer can lose a whole season to one bad decision about what to plant or when to water. Soil testing is not always nearby, extension officers have too many villages to cover, and the advice that does arrive is often hard to act on. Meanwhile, years of heavy fertiliser and pesticide use have worn out soils and polluted water in many regions, which is one reason more growers are turning to organic methods [26], [27]. Going organic means more than skipping chemicals. The grower has to read the soil, rotate crops, build fertility with compost and manure, and control pests by biological and mechanical means. All of that needs knowledge that many farmers cannot easily get.

Machine learning has started to help here. Surveys report growing use of it for yield prediction, disease recognition and more [14], [15], [19]. The trouble is that most tools we came across do a single job. One suggests a crop, another labels a sick leaf, and hardly any consider whether the inputs are organic. We would like one place where a farmer can go from asking what to plant, to asking why the leaves are spotted, to asking whether to irrigate tomorrow, without changing apps or reading jargon.

A. Problem statement

Farmers who want to follow organic practice have no easy, low-cost source of guidance that ties together soil condition, weather, plant health and water use. The digital tools that exist are scattered, mostly written for conventional farming, and rarely designed for people with little technical background.

B. Objectives

We have five aims. The system should (1) suggest organic crops and natural fertilisers from soil and climate data, (2) recognise plant diseases from leaf photos and propose organic treatments, (3) give irrigation advice that takes the forecast into account, (4) answer questions through a chatbot in simple language, and (5) encourage practices that protect soil and water.

C. Contributions and organisation

We claim three contributions. First, an architecture that brings four advisory functions under one interface. Second, an organic knowledge layer that converts raw model output into treatments a farmer following organic rules can use. Third, an evaluation plan that covers model quality as well as usability. Section II reviews related work, Section III describes the system design, Section IV explains each module, Section V covers data and implementation, Section VI gives the evaluation plan, Section VII discusses deployment, ethics and limitations, and Section VIII concludes.

II. RELATED WORK

A. Crop recommendation and tabular learning

Random Forests [1] are a popular choice for crop recommendation on tabular data. They cope well with the messy, non-linear links between soil and weather variables, and they overfit less than many alternatives. Gradient-boosted trees such as XGBoost [8] often do even better on structured data, so we treat them as a natural point of comparison. Libraries like scikit-learn [9] make it easy to train and compare these models. In their review of machine learning in agriculture, Liakos et al. [15] list crop management, water management and soil management among the main application areas.

B. Plant disease detection

Convolutional networks changed image-based diagnosis. Early large networks [12] and deeper designs such as VGG [24] and ResNet [7] set the pace for image recognition. Mohanty et al. [2] trained deep networks on the PlantVillage collection [3] and reported strong

accuracy, though on clean, lab-style photographs. Ferentinos [16] compared several architectures for plant disease diagnosis on a large image database and also reported high success rates. Compact models such as MobileNet [4] and MobileNetV2 [6] cut the computation needed, which makes it realistic to run them on modest hardware. Transfer learning [18] lets us begin with features learned on ImageNet [11] rather than train from scratch, which matters when labelled farm images are scarce, and data augmentation [17] adds further robustness.

C. Irrigation and water management

The FAO-56 guidelines [5] are still the standard reference for estimating how much water a crop loses through evapotranspiration. The method multiplies a reference evapotranspiration value by a crop coefficient that changes with growth stage. If forecast data feed into this calculation, irrigation can be planned around expected rain instead of a fixed calendar.

D. Conversational assistants and smart farming

Transformer-based language models [21], [20] have made conversational assistants far more capable, and retrieval-augmented generation [22] shows how answers can be tied to a trusted set of documents. That idea fits advisory work well, since a wrong answer here can cost a farmer money. Wider surveys of smart farming [19] also note that adoption depends as much on how easy a tool is to use as on how accurate its models are.

E. Organic standards and decision support

Organic production follows standards that restrict synthetic inputs and require practices such as crop rotation and organic matter management [27]. A decision-support tool that ignores those rules may recommend something a certified organic farmer is not allowed to use. Comparisons of organic and conventional yields [26] show that results depend a great deal on the crop and on how well the farm is managed, which argues for advice tailored to local conditions instead of generic recommendations. The review of big data in smart farming [19] makes a related point: many promising tools never reach ordinary farmers because they are hard to use or ignore local conditions.

F. Gap addressed

What we add is not a new algorithm. It is the combination: these pieces working together, tuned toward organic practice and wrapped in an interface a non-specialist can use. Table I summarises the inputs the system works with.

TABLE I. INPUTS USED BY THE ADVISORY SYSTEM.

Parameter	Unit / type	Role in the system
Nitrogen (N)	soil test unit, e.g. kg/ha	Crop suitability; deficiency triggers compost or green-manure advice
Phosphorus (P)	soil test unit, e.g. kg/ha	Crop suitability; deficiency triggers phosphorus-rich organic amendments
Potassium (K)	soil test unit, e.g. kg/ha	Crop suitability; deficiency triggers potassium-rich organic amendments
Soil pH	unitless	Crop suitability; very acidic or alkaline soil triggers correction advice
Temperature	degrees Celsius	Crop suitability; evapotranspiration estimate
Humidity	percent	Crop suitability; disease-risk context
Rainfall	millimetres	Crop suitability; irrigation deferral
Leaf image	JPEG / PNG photo	Input to the disease detector
Location	district or coordinates	Weather forecast lookup

III. SYSTEM DESIGN

The application has three layers. Farmers use a mobile-friendly web page. Behind it a Python API runs the models and the advice logic, and underneath sit the datasets, a small knowledge base of organic treatments, and each farmer's history. Weather data comes from an outside forecast service. Figure 1 shows how the parts fit together.

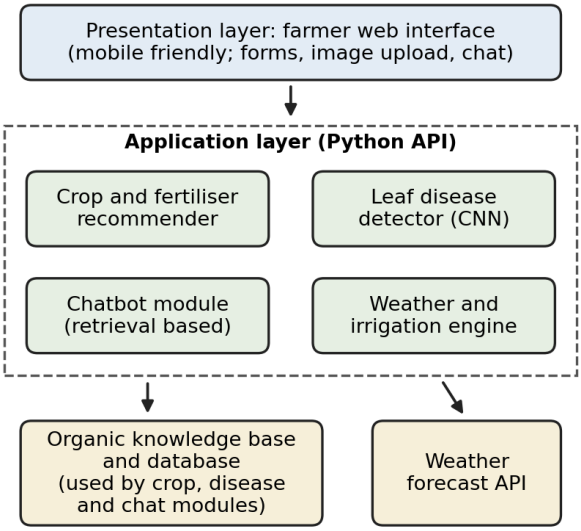


Fig. 1. Overall architecture of the proposed advisory system.

A. Design principles

We followed four principles when designing the system. The first is simplicity: every screen asks for as little as possible and avoids technical terms. The second is organic first: each recommendation passes through the knowledge base so that a farmer is never told to use a prohibited input. The third is transparency: disease predictions come with a confidence value, and the system says when it is unsure. The fourth is light weight: the models are small enough to run on a modest server and answer within a few seconds.

B. User flow

A farmer registers, sets a location and picks one of four options: crop advice, disease check, irrigation advice or chat. Each option opens a short form or an upload screen and ends on a result page with a plain explanation. Results are saved so the farmer can look back at earlier advice, and the system can avoid repeating a treatment that has already been tried.

C. Database design

A relational database stores farmer accounts, farm details, past queries and generated advice, while the organic knowledge base is kept as structured records that the recommendation and chatbot modules both read. Keeping a single knowledge source avoids contradictory advice between modules. The main entities are described below. Images are stored on disk with only their paths and prediction results kept in the database, which keeps queries fast and the database small.

The main entities are the farmer (identifier, name, preferred language, location), the farm (area, soil type, current crop), soil records (N, P, K, pH and date), disease queries (image path, predicted class, confidence and date), an advice log, and knowledge entries (topic, condition, organic option and caution). Each module reads from or writes to a small subset of these entities.

D. Requirements

Functional requirements include account registration, soil data entry, image upload, forecast retrieval, chat interaction and history viewing. Non-functional requirements are equally important: pages should load quickly on slow connections, the interface should be usable with minimal reading, advice should be consistent across modules, and personal data should be protected. Accessibility features such as large buttons, icons alongside text and, in future, voice input would help users with limited literacy.

IV. METHODOLOGY

A. Crop and fertiliser recommendation

The farmer types in nitrogen, phosphorus, potassium, pH, temperature, humidity and rainfall. After the data are cleaned and scaled, a classifier picks the best-matching crop. Figure 2 shows the whole flow. We will start with Random Forest and compare it with Decision Tree, Naive Bayes, SVM and XGBoost. The shortlist is then cut down to crops suited to organic cultivation. When the soil is short of a nutrient, simple rules point to natural fixes, and Table II gives examples of the mapping we plan to store in the knowledge base.

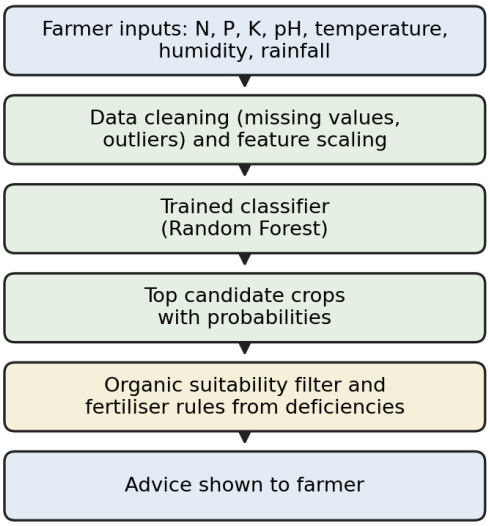


Fig. 2. Data flow of the crop and fertiliser recommendation module.

TABLE II. EXAMPLES OF ORGANIC RECOMMENDATIONS IN THE KNOWLEDGE BASE (TO BE REVIEWED BY AN AGRONOMIST).

Condition	Natural option	Notes
Low nitrogen	Compost, vermicompost, farmyard manure, legume green manure	Legume cover crops also add nitrogen through fixation
Low phosphorus	Bone meal, rock phosphate, phosphorus-solubilising biofertilisers	Slow acting; apply before sowing
Low potassium	Wood ash, well-rotted compost, potassium-rich plant residues	Avoid overuse of ash on alkaline soil
Acidic soil (low pH)	Agricultural lime, wood ash	Test pH again after a few weeks
Powdery mildew	Neem-oil spray, diluted buttermilk spray	Spray in cooler hours of the day
Sucking pests (aphids)	Neem-oil or soap spray, encourage natural predators	Repeat as needed
Root rot / wilt	*Trichoderma*-based biocontrol, crop rotation, better drainage	Remove severely infected plants
Fungal leaf spot	Remove infected leaves, rotation, adequate spacing	Avoid overhead watering late in the day

B. Disease detection from leaf photos

The farmer photographs the affected leaf and uploads it. The image is resized and normalised, and during training we add flips, rotations and brightness changes so that the model does not depend on a single lighting condition. The network starts from a model pre-trained on ImageNet (MobileNetV2 or ResNet) and is fine-tuned on a leaf-disease dataset such as PlantVillage [3], using the Adam optimiser [25] and cross-entropy loss. For an image with feature vector z over C classes, the softmax probability of class i is

$$p_i = exp(z_i) / \Sigma_{j=1..C}\, exp(z_j),$$

and the loss for a true class y is $L = -\log p_y$. The farmer sees the disease name, how confident the model is, and an organic option from the knowledge base. If the top probability falls below a chosen threshold, the app asks for a clearer photo or suggests seeing an expert instead of guessing. Figure 3 outlines the pipeline. A visual

explanation method such as Grad-CAM [28] could also be added to show which leaf regions drove a prediction.

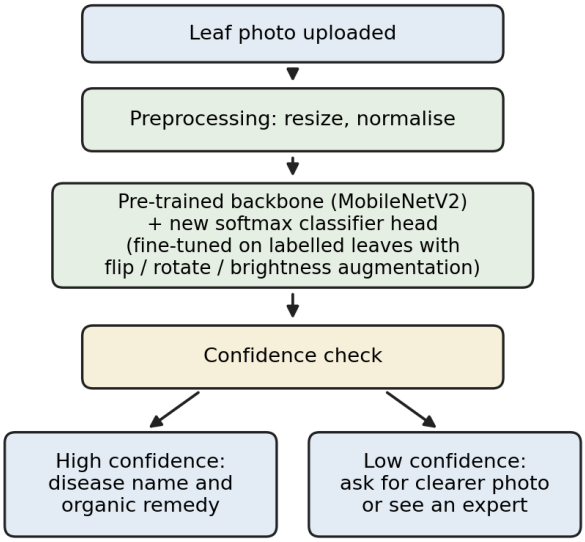


Fig. 3. Leaf disease detection pipeline with a confidence check.

C. Weather and irrigation

We fetch temperature, humidity, wind and rain probability for the farmer's location. Following FAO-56 [5], crop water need is estimated as

$$ET_c = K_c \times ET_0,$$

where ET_0 is the reference evapotranspiration and K_c is a crop coefficient that depends on the crop and its growth stage. The irrigation requirement is then roughly the crop water need minus effective rainfall and the water already in the soil. If heavy rain is on the way, the app tells the farmer to wait and save water. If a dry spell is ahead, it suggests when and how often to irrigate. Figure 4 shows the decision logic.

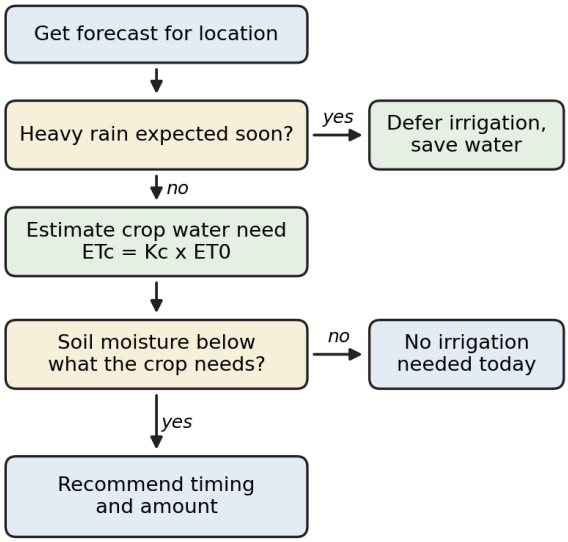


Fig. 4. Decision flow of the weather-aware irrigation module.

D. Chatbot

The chatbot handles questions about crops, composting, pests and use of the app, and Figure 5 shows its flow. Answers are drawn from our organic knowledge base so that replies stay grounded [22], and they are written to sound like a helpful neighbour rather than a textbook. A first version can use intent matching with templates. A later version could use a language model [20], [21] restricted to answer only from retrieved passages. Regional-language support is something we would like to add.

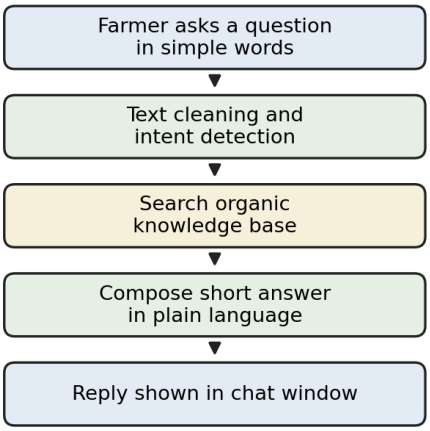


Fig. 5. Retrieval-based chatbot workflow.

E. Model selection and tuning

For the tabular recommender we will compare candidate models under the same cross-validation folds so that differences reflect the algorithm rather than the split. Hyperparameters such as the number of trees, maximum depth and minimum samples per leaf will be tuned by grid or random search. Feature importance from the Random Forest will be examined to confirm that the model relies on sensible variables, for example rainfall and nitrogen for crops with known water and nutrient demands. For the image model, the learning rate, number of unfrozen layers, batch size and augmentation strength are the main settings to tune, and early stopping on a validation set will be used to avoid overfitting.

F. Building the knowledge base

The knowledge base is built from agricultural extension publications, university guides and organic certification standards. Each entry links a condition (a nutrient deficiency, a disease or a question topic) to a permitted option, a short method of use and a caution. Entries are written in plain language and reviewed by an agronomist before being released. Version numbers allow older advice to be traced if a recommendation is later revised.

G. Illustrative walkthrough

The following scenario is hypothetical and is meant only to show how the modules cooperate; it does not report experimental results. A farmer in a semi-arid district enters a soil report showing low nitrogen and slightly acidic pH, together with recent temperature, humidity and rainfall. The recommender returns a short list of crops, removes those that are unsuitable for organic cultivation, and attaches advice such as adding compost and a legume

green manure. A few weeks later the farmer notices spots on leaves, uploads a photo, and receives a predicted disease with a confidence value and an organic treatment. Before spraying, the farmer opens the irrigation page; the forecast shows rain in two days, so the app advises postponing watering. Finally, the farmer asks the chatbot how long to wait before harvesting after a neem-oil spray, and receives an answer drawn from the knowledge base.

H. Algorithm summary

At a high level the system follows four steps for each request: (1) validate and clean the input; (2) call the relevant model or rule engine; (3) pass the raw output through the organic knowledge base to obtain permitted, understandable advice; and (4) store the result and present it in the farmer's language with a note on confidence or uncertainty. Separating model output from final advice also means that the knowledge base can be updated by experts without retraining any model.

V. DATA AND IMPLEMENTATION

A. Datasets

For crop recommendation we plan to use a publicly available dataset that links soil and climate measurements to suitable crops (the commonly used version has 2,200 records covering 22 crops). For disease detection we plan to use PlantVillage [3], which contains tens of thousands of labelled leaf images across many crop-disease classes. Both datasets have limits. The crop data are not drawn from a single region, and PlantVillage images are taken against plain backgrounds. We therefore plan to collect a small set of field photographs to test how well the detector generalises.

B. Preprocessing and training

Tabular data are checked for missing values and implausible readings, then scaled. Class balance is inspected, and if some crops are rare, techniques such as SMOTE [23] can be considered. The data are split into training and test sets, with cross-validation used to choose hyperparameters. Images are resized to the input size of the chosen network, normalised, and augmented. Fine-tuning is done in two stages: first only the new classifier head is trained, then some deeper layers are unfrozen with a small learning rate.

C. Technology stack

TABLE III. PLANNED TECHNOLOGY STACK.

Layer	Tools we plan to use
Front end	HTML, CSS and JavaScript, designed for phones first
Back end	Python with Flask or FastAPI
Machine learning	scikit-learn [9], TensorFlow / Keras [10]
Data	CSV files, an image dataset, an SQL database
Outside services	A weather forecast API
Version control	Git and GitHub

D. Interface and API design

The back end exposes a small set of endpoints: one to submit soil and climate values and receive crop and fertiliser advice, one to accept a leaf image and return the predicted disease, one to request irrigation advice for a farm and date, and one to send a chat message and receive a reply. Each endpoint validates its input, returns a short structured response, and records the request in the advice log. On the front end, each function has its own page with large buttons, short labels and icons, so a first-time user can reach any feature in one or two taps. Results are shown as a few plain sentences followed by details that can be expanded.

E. Runtime environment

The prototype is designed to run on a modest cloud instance or a laptop. Trained models are saved to disk and loaded once when the server starts, so each request only performs inference. The tabular model is small and answers almost instantly; the image model is a lightweight network and should respond within a few seconds on a standard processor. Image uploads are limited in size, resized on the client where possible, and deleted after processing unless the farmer chooses to keep them for history. A configuration file holds API keys and settings so that no secrets are stored in the source code.

F. Version control and reproducibility

All code, configuration and trained model files are kept under version control, and random seeds are fixed during training so that results can be reproduced. The datasets used, their versions, the split used for testing and the main hyperparameters are recorded alongside each trained model. This record makes it possible to compare experiments fairly and to explain exactly how any figure in the final results table was obtained.

VI. EVALUATION PLAN

Both models will be tested on data they have not seen, using an 80/20 split and cross-validation. We will report accuracy, precision, recall and F1-score [29], with confusion matrices to show where the models get confused. For a class, precision is TP / (TP + FP), recall is TP / (TP + FN), and F1 is their harmonic mean; macro averages will be used so that rare classes count equally. The irrigation advice will be checked against standard water-requirement calculations for sample cases. Response time and basic load behaviour of the web application will also be measured.

We also want real feedback: a short survey with farmers or agriculture students on whether the app is easy to use, whether the advice is clear, and whether they would trust it. Table IV gives the template for reporting model results; the values are left blank until experiments are finished.

TABLE IV. RESULTS TEMPLATE, TO BE FILLED IN AFTER TRAINING AND TESTING.

Module	Model	Acc.	Prec.	Rec.	F1
Crop recommendation	Random Forest	—	—	—	—
Crop recommendation	Decision Tree	—	—	—	—
Crop recommendation	SVM / XGBoost	—	—	—	—
Disease detection	MobileNetV2 (fine-tuned)	—	—	—	—
Disease detection	ResNet (fine-tuned)	—	—	—	—

C. Usability study design

To judge whether the system is genuinely helpful, we plan a small usability study with agriculture students and, where possible, local farmers. Participants will complete a fixed set of tasks: entering soil data to obtain a crop suggestion, uploading a leaf photo, checking irrigation advice and asking two questions in the chatbot. We will record task completion, time taken and the number of errors, and ask participants to rate ease of use, clarity of advice and trust on a five-point scale. Open comments will be collected to identify confusing wording or screens. Results will be used to revise the interface before any wider release.

D. Comparisons and ablation

Model comparison will include the baseline classifiers listed above. For the disease detector we will also compare training from scratch with transfer learning, and with and without augmentation, to show how much each choice contributes. For the full system, we will compare the connected workflow against a version in which the modules are used separately, measuring the time a participant needs to complete a combined task such as diagnosing a disease and scheduling treatment. These comparisons keep the claims of the paper tied to evidence.

E. Testing plan

Besides model metrics, the web application needs functional testing. The main test cases we intend to run before field trials are described here, with expected behaviour; actual outcomes will be recorded during testing.

The main cases are: valid soil data should produce a crop list and fertiliser advice; a missing value should trigger a clear request for it; an out-of-range value such as a pH of 20 should be rejected with an explanation; a clear leaf photo should return a class, confidence and remedy; a blurry or non-leaf image should produce a low-confidence message and a request to retake the photo; a high rain probability should defer irrigation; an off-topic chat message should receive a polite statement that the bot cannot help; and a throttled network connection should still load the page and show upload progress.

F. Threats to validity

Several factors could weaken the conclusions of the evaluation. Public datasets may not reflect the soils, climates and photographs of the target region, so accuracy on them may overstate real-world performance. Small user studies can be influenced by the enthusiasm of participants or by differences in their prior knowledge. Test splits that share near-duplicate images between training and testing could inflate disease-detection scores, so we will split by plant or by source where the data allow. Finally, agronomic advice depends on local conditions, so expert review of the knowledge base remains essential. We will report these limits openly alongside the numbers.

VII. DISCUSSION

A. Expected benefits

If the system works as intended, a farmer could choose crops that fit the land, use fewer chemical inputs, catch disease sooner and waste less water, all from one screen. Having a single assistant tie everything together should also make the advice feel consistent. Comparisons of organic and conventional yields [26] suggest that organic systems can compete under good management, and good management is exactly where sound advice counts.

B. Deployment considerations

The platform can be hosted on a small cloud server. Because many users will be on phones with limited data, pages should be light and images compressed before upload. A later step is to cache the last forecast and advice so that the app remains partly useful offline. Support from agricultural extension workers would help during early field trials.

C. Ethics, privacy and responsibility

The system stores location and farm information, so it should collect only what it needs, protect it, and explain how it is used. Advice must never be presented as a guarantee. Organic treatment suggestions should be reviewed by agronomists, and the chatbot should say plainly when it does not know. These steps matter because farmers may act directly on what the app says.

D. Limitations

There are real limits. Models trained on tidy lab photos often do worse on field images, where soil, shadows and neighbouring leaves clutter the background. The advice is only as good as the soil data the farmer enters, and one reading may not represent a whole field because nutrient levels vary. Forecasts can be wrong. And in many villages the internet connection is patchy.

E. Security, privacy and trust

Farm location, soil data and photographs are personal information. The platform should use encrypted connections, hash stored passwords, limit each module to the data it needs, and let farmers delete their history. Trust also depends on honesty about uncertainty: confidence values, clear wording such as "likely" rather than "certain", and a visible suggestion to consult an expert for severe or unclear cases. Because farmers may act directly on the advice, every recommendation page will carry a short note that it is decision support and not a guarantee.

F. Risks and mitigation

Several risks deserve attention. Dataset bias may make the models weaker for crops or regions that are under-represented; this can be reduced by collecting local samples and reporting per-class results. Wrong or outdated knowledge-base entries can mislead users; expert review and version tracking address this. Dependence on an external weather service creates a point of failure; caching the last forecast and showing its age mitigates it. Finally, low digital literacy may limit adoption; short tutorials, icons, regional languages and help from local extension workers can improve uptake.

G. Scalability and maintenance

The modular design allows each component to be updated independently: a better disease model can replace the current one without touching the recommender, and new crops or treatments can be added by editing the knowledge base. If usage grows, the API can be containerised and scaled horizontally, while model inference for images can be moved to a separate worker so that long requests do not block the website. Logging of anonymised errors and user feedback will guide continuous improvement.

H. Sustainability impact

The system supports sustainability in three ways. First, nutrient advice that favours compost, manures and green manures reduces dependence on synthetic fertilisers and helps rebuild soil organic matter. Second, early disease identification allows targeted organic treatment, which limits the spread of infection and the quantity of spray applied. Third, forecast-aware irrigation avoids watering just before rain, saving water and energy for pumping. These benefits are expected rather than measured, and field trials would be needed to quantify them.

I. Comparison with single-purpose tools

Single-purpose tools usually focus on one task and require the farmer to interpret the output alone. Our framework differs in that its outputs share one knowledge base, one language setting and one history. A farmer who receives a disease prediction can immediately ask the chatbot how to prepare the remedy, or check the irrigation page to time the application. This connected workflow, rather than any single model, is the main practical advantage we aim to demonstrate in user testing.

J. Future enhancements

Several extensions would make the platform more useful. Low-cost soil sensors could send nitrogen, moisture and pH readings directly, removing manual entry and the errors it brings. A mobile application with offline mode would serve areas with poor coverage, storing questions and photos until a connection returns. Voice interaction in regional languages would help farmers who find typing difficult. Yield prediction and a harvest calendar could use the same data to help plan labour and storage, and a market-price module could suggest when and where to sell. Community features, such as letting nearby farmers share verified experiences, could add local knowledge that no dataset captures.

K. Expected contribution

The main contribution we expect is practical rather than algorithmic: a tested design that shows how existing machine learning techniques can be combined with expert-reviewed organic knowledge into one usable service. If the planned experiments confirm good accuracy and positive user feedback, the design can serve as a starting point for agricultural institutions, student projects and non-profit organisations that want to offer similar advice in their own regions.

VIII. CONCLUSION AND FUTURE WORK

We have presented a plan for a single web platform that combines crop and fertiliser recommendation, leaf disease detection, irrigation advice and a chatbot, all geared to organic farming. The design keeps the interface simple, sends every recommendation through an organic knowledge base and shows uncertainty openly. Next steps include low-cost soil sensors that feed readings in automatically, an offline mobile version, voice support in regional languages, training on real field photos and, later, yield prediction and market prices. The experiments described in Section VI will show how well the design works in practice.

The work also has wider implications. If a single, low-cost platform can give small farmers reliable organic guidance, it could support the shift toward more sustainable food production while keeping decisions in the hands of the people who know the land best. Equally important, the design principles used here, namely routing model output through an expert-reviewed knowledge base, reporting uncertainty openly and keeping the interface simple, can be reused in other advisory domains such as livestock care or post-harvest storage.

ACKNOWLEDGEMENT

The authors thank their project guide, S. Anitha, Assistant Professor, and the Department of Artificial Intelligence and Machine Learning, Anurag University, for their guidance and support.

REFERENCES

[1] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[2] S. P. Mohanty, D. P. Hughes and M. Salathé, “Using deep learning for image-based plant disease detection,” *Frontiers in Plant Science*, vol. 7, 2016.

[3] D. P. Hughes and M. Salathé, “An open access repository of images on plant health to enable the development of mobile disease diagnostics,” arXiv:1511.08060, 2015.

[4] A. G. Howard et al., “MobileNets: Efficient convolutional neural networks for mobile vision applications,” arXiv:1704.04861, 2017.

[5] R. G. Allen, L. S. Pereira, D. Raes and M. Smith, “Crop evapotranspiration: Guidelines for computing crop water requirements,” FAO Irrigation and Drainage Paper 56, 1998.

[6] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov and L.-C. Chen, “MobileNetV2: Inverted residuals and linear bottlenecks,” in *Proc. IEEE CVPR*, 2018, pp. 4510–4520.

[7] K. He, X. Zhang, S. Ren and J. Sun, “Deep residual learning for image recognition,” in *Proc. IEEE CVPR*, 2016, pp. 770–778.

[8] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proc. ACM SIGKDD*, 2016, pp. 785–794.

[9] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830,

2011.

[10] M. Abadi et al., “TensorFlow: A system for large-scale machine learning,” in *Proc. USENIX OSDI*, 2016, pp. 265–283.

[11] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li and L. Fei-Fei, “ImageNet: A large-scale hierarchical image database,” in *Proc. IEEE CVPR*, 2009, pp. 248–255.

[12] A. Krizhevsky, I. Sutskever and G. E. Hinton, “ImageNet classification with deep convolutional neural networks,” in *Advances in Neural Information Processing Systems*, vol. 25, 2012.

[13] Y. LeCun, Y. Bengio and G. Hinton, “Deep learning,” *Nature*, vol. 521, pp. 436–444, 2015.

[14] A. Kamilaris and F. X. Prenafeta-Boldú, “Deep learning in agriculture: A survey,” *Computers and Electronics in Agriculture*, vol. 147, pp. 70–90, 2018.

[15] K. G. Liakos, P. Busato, D. Moshou, S. Pearson and D. Bochtis, “Machine learning in agriculture: A review,” *Sensors*, vol. 18, no. 8, p. 2674, 2018.

[16] K. P. Ferentinos, “Deep learning models for plant disease detection and diagnosis,” *Computers and Electronics in Agriculture*, vol. 145, pp. 311–318, 2018.

[17] C. Shorten and T. M. Khoshgoftaar, “A survey on image data augmentation for deep learning,” *Journal of Big Data*, vol. 6, art. 60, 2019.

[18] S. J. Pan and Q. Yang, “A survey on transfer learning,” *IEEE Trans. Knowledge and Data Engineering*, vol. 22, no. 10, pp. 1345–1359, 2010.

[19] S. Wolfert, L. Ge, C. Verdouw and M.-J. Bogaardt, “Big data in smart farming – A review,” *Agricultural Systems*, vol. 153, pp. 69–80, 2017.

[20] J. Devlin, M.-W. Chang, K. Lee and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.

[21] A. Vaswani et al., “Attention is all you need,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[22] P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020.

[23] N. V. Chawla, K. W. Bowyer, L. O. Hall and W. P. Kegelmeyer, “SMOTE: Synthetic minority over-sampling technique,” *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.

[24] K. Simonyan and A. Zisserman, “Very deep convolutional networks for large-scale image recognition,” in *Proc. ICLR*, 2015 (arXiv:1409.1556).

[25] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *Proc. ICLR*, 2015 (arXiv:1412.6980).

[26] V. Seufert, N. Ramankutty and J. A. Foley, “Comparing the yields of organic and conventional agriculture,” *Nature*, vol. 485, pp. 229–232, 2012.

[27] J. P. Reganold and J. M. Wachter, “Organic agriculture in the twenty-first century,” *Nature Plants*, vol. 2, art. 15221, 2016.

[28] R. R. Selvaraju et al., “Grad-CAM: Visual explanations from deep networks via gradient-based localization,” in *Proc. IEEE ICCV*, 2017, pp. 618–626.

[29] M. Sokolova and G. Lapalme, “A systematic analysis of performance measures for classification tasks,” *Information Processing & Management*, vol. 45, no. 4, pp. 427–437, 2009.